

Homework 11: A Typed Cache

Build a generic container, run into three erasure limits on purpose, and cause heap pollution so you can see exactly where the blame lands.

Part One: The Cache

Create `TypedCache.java` in the classic form.

Write a `Cache<K, V>` with a fixed capacity:

- `put(K key, V value)` — when full, evict the **oldest** entry before inserting
- `get(K key)` — returns the value or `null`
- `size()` and `keys()`

Show it working with `Cache<String, Integer>`, prove eviction happens, then use the same class as `Cache<Integer, List<String>>` to show one implementation serving unrelated types.

Hit The Limits

While writing it, try each of these, confirm it does not compile, then solve it another way. Document all three in a comment on the class:

- Store the values in a `V[]` array
- Validate with `if (value instanceof V)`
- Share a fallback via `static V defaultValue;`

For each, say what the error was and what you did instead.

Part Two: Bounds and PECS

Write three methods:

Method	Signature shape
<code>largest(List<T>)</code>	needs a bound so it can call <code>compareTo</code>
<code>average(...)</code>	reads numbers, never writes
<code>drainInto(...)</code>	writes integers, never reads

Give `average` and `drainInto` the correct wildcards. Prove `average` works on both a `List<Integer>` and a `List<Double>`, and that `drainInto` works into both a `List<Number>` and a `List<Object>`.

Explain It

Write a comment covering, in your own words:

- Why `largest` cannot work without its bound
- Why `average` uses one wildcard and `drainInto` the other
- What each wildcard **gives up**, and why giving that up is what makes it safe

Answers that just restate "producer extends, consumer super" do not count. Say what the compiler would refuse to let you do in each case.

Part Three: Pollute The Heap

Build a `List<String>`, assign it to a **raw** `List`, and add an `Integer` through the raw reference.

Then print, in order:

1. The list's contents
2. Element 0, read as a `String`
3. Element 1, read as a `String`, catching what happens

The Question That Matters

Write a comment answering: **which line broke the invariant, and which line threw?**

Then explain why they are different lines, what the compiler inserted where, and why this is harder to debug than a normal crash.

Acceptance Criteria

- ☐ Runs with `java TypedCache.java`
- ☐ `Cache<K, V>` is generic over both parameters and evicts the oldest entry
- ☐ The same `Cache` class is used with two unrelated type argument pairs
- ☐ A comment documents all three erasure limits and the workaround used
- ☐ `largest` declares a bound, with a comment on why it is required
- ☐ `average` and `drainInto` use the correct wildcards
- ☐ Both are proven against two different element types
- ☐ The explanation names what each wildcard forbids
- ☐ Heap pollution is reproduced and the `ClassCastException` caught
- ☐ A comment identifies the offending line and the throwing line as different

Stretch

Make `Cache` evict the **least recently used** entry rather than the oldest inserted, so a `get` counts as a use.

`LinkedHashMap` can do this for you with one constructor argument and one overridden method. Find it, use it, and write a comment on what you would have had to write by hand otherwise.

Hint, if part three does not throw

Check that you are reading element 1 into a `String` variable, not into `var` or `Object`. The cast is inserted because of the **declared type at the read site**. Read it as `Object` and there is no cast, so there is no failure, and the polluted list sits there quietly until someone else reads it properly.